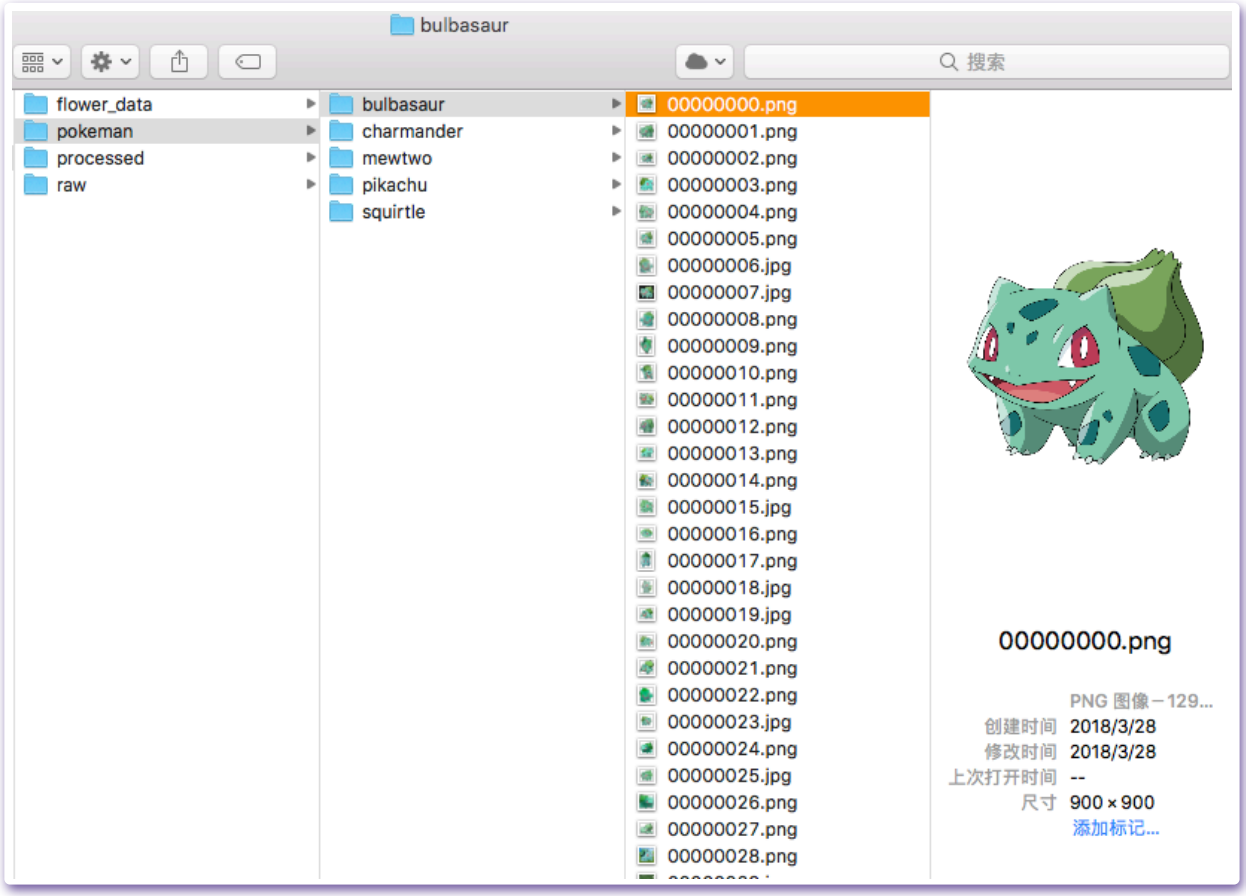
1 一、自定义数据集

1.1 pytorch默认形式

1.1.1 pytorch二级目录形式

- 结构如下图,每个目录是一个分类，每个目录下面保存当前分类的所有图片



1.1.2 直接读取

- 可直接使用 `ImageFolder` 读取

```
import torchvision
from torchvision import datasets,transforms
from torch.utils.data import DataLoader

tf = transforms.Compose([
    transforms.Resize((224,224)),
    transforms.ToTensor()
])

db = datasets.ImageFolder(root='../data/pokeman/',transform=tf)
loader = DataLoader(db,batch_size=32,shuffle=True)

```

```
x,y=next(iter(loader))

print(x.shape)
print(y.shape)
print(y)



"""输出"""
torch.Size([32, 3, 224, 224])
torch.Size([32])
tensor([1, 0, 1, 1, 2, 2, 1, 1, 0, 1, 3, 4, 3, 4, 3, 3, 0, 4, 4, 1, 0, 1, 1, 2,
        2, 4, 1, 3, 1, 1, 0, 2])

```

1.1.3 自定义读取

- 需要继承自`torch.utils.data.DataSet`类，训练，验证，测试分别为60%，20%，20%

```

import torch
import os,glob
import random,csv
from torch import nn
from torch.nn import functional as F
import torchvision
from torchvision import models,datasets,transforms
from torch.utils.data import Dataset,DataLoader
from PIL import Image
import visdom
import time


class Pockmon(Dataset):
    def __init__(self,root,resize,mode):
        """__init__:   初始化函数

        Parameters:
        ------------------------------------------------------------------------------------------------------------
        root:               type:str                desc:图片根目录,存储格式：跟目录下的目录则是类别标签,里面存放当前类别标签的所有图片

        resize:             type:int                desc:

        mode:               type:str                desc:

        """
        super(Pockmon,self).__init__()
```

```
        self.root = root
        self.resize = resize

        self.name2label = {}  # 将类别标签依次转为数字
        for name in sorted(os.listdir(os.path.join(root))):#每次返回的顺序可能不一致,所以先sort
            if not os.path.isdir(os.path.join(root,name)): # 如果不是目录,则过滤
                continue
            self.name2label[name] = len(self.name2label.keys())

        # print(self.name2label)

        # load image path and label list
        self.images,self.labels = self.load_csv('images.csv') # images 和 labels ——对应

        if mode == "train": # 60%训练
            self.images = self.images[:int(0.6*len(self.images))]
            self.labels = self.labels[:int(0.6*len(self.labels))]
        elif mode in  ["val",'valid']: # 20%验证
            self.images = self.images[int(0.6*len(self.images)):int(0.8*len(self.images))]
            self.labels = self.labels[int(0.6*len(self.labels)):int(0.8*len(self.labels))]
        elif mode == 'test': # 20%测试
            self.images = self.images[int(0.8*len(self.images)):]
            self.labels = self.labels[int(0.8*len(self.labels)):]

        else:
            raise('mode must be in [train,val,valid,test]')


    def load_csv(self,filename):
        """load_csv:  将图片,标签存储为csv格式,返回所有的图片地址列表和图片标签列表
                      如果filename存在,则直接读取并返回
        Parameters:
        ------------------------------------------------------------------------------------------------------------
        filename:               type:str                desc: 图片地址和图片标签csv文件文件名

        Returns:
        ------------------------------------------------------------------------------------------------------------
        images:                 type:list               desc: 所有的图片地址列表
        labels:                 type:list               desc: 所有的图片类别标签列表
        """

```

```
        if not os.path.exists(os.path.join(self.root,filename)): # 如果这个文件不存在,则创建,否则不重复创建
            images = []
            for name in self.name2label.keys(): # 将当前目录下所有的文件合并到列表中去,下面是包含了三种不同种类的文件
                images += glob.glob(os.path.join(self.root,name,'*.png'))
                images += glob.glob(os.path.join(self.root,name,'*.jpg'))
                images += glob.glob(os.path.join(self.root,name,'*.jpeg'))
            # 'E:\\数据\\pokemon\\bulbasaur\\00000000.png
            # print(len(images),images)

            random.shuffle(images)
            # 存入csv文件
            with open(os.path.join(self.root,filename),mode='w',newline='') as f:
                writer = csv.writer(f)
                for img in images: #'E:\\数据\\pokemon\\bulbasaur\\00000000.png
                    name = img.split(os.sep)[-2] # bulbasaur
                    label = self.name2label[name]
                    # E:\\数据\\pokemon\\bulbasaur\\00000000.png, label
                    writer.writerow([img,label])
                print('Writen into csv file:',os.path.join(self.root,filename))

        # 读取csv文件
        images,labels = [],[]
        with open(os.path.join(self.root,filename)) as f:
            reader = csv.reader(f)
            for row in reader:
                # E:\\数据\\pokemon\\bulbasaur\\00000000.png, label
                img,label = row
                label = int(label)

                images.append(img)
                labels.append(label)
        assert len(images) == len(labels)
        return images,labels


    def __len__(self):
        return len(self.images)

    def __getitem__(self, idx):
        # idx 范围[0-len(self.images)]
        # self.images,self.labels
        # img     :   E:\\数据\\pokemon\\bulbasaur\\00000000.png,
```

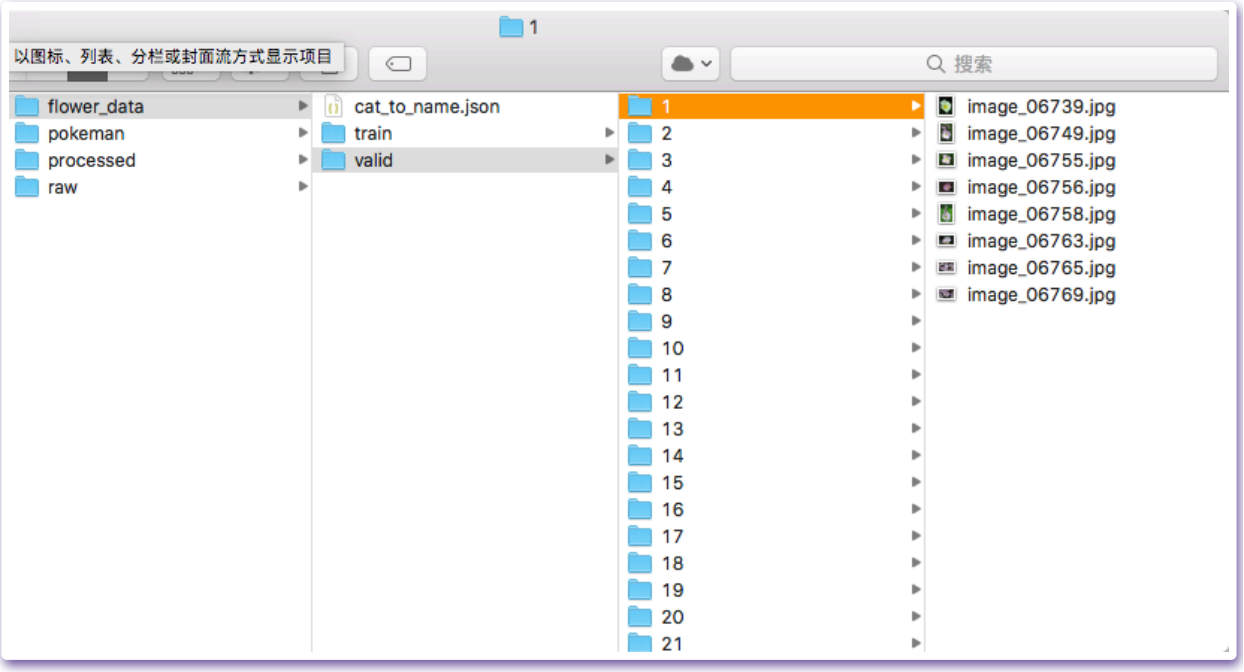
```python
        # label  :   类别数字
        img,label = self.images[idx],self.labels[idx]
        tf = transforms.Compose([
            lambda x:Image.open(x).convert('RGB'), # strin_path => image_data, convert是png会多一个透明通道，为4通道
            # transforms.Resize((self.resize,self.resize)),
            transforms.Resize((int(self.resize * 1.25),int(self.resize * 1.25))),
            transforms.RandomRotation(15), # 随机旋转角度如果太大， 可能导致网络不收敛
            transforms.CenterCrop(self.resize), # 随机裁剪之后，旋转后的图片可能会有阴影留白，所以需要做一下中心裁剪
            transforms.ToTensor(),
            transforms.Normalize(mean=[0.485,0.456,0.406]
                                 ,std=[0.229,0.224,0.225])
        ])

        img = tf(img)
        label = torch.tensor(label)
        print(label)
        return img,label

    def denormalize(self,x_hat):
        """denormalize:   将已经normalize的图片tensor复原
        """
        mean=[0.485,0.456,0.406]
        std=[0.229,0.224,0.225]

        # x:[c,h,w]
        # mean: [3]=>[3,1,1]
        mean = torch.tensor(mean).unsqueeze(1).unsqueeze(1)
        std = torch.tensor(std).unsqueeze(1).unsqueeze(1)
        x = x_hat * std + mean

        return x


def main():
    viz = visdom.Visdom()
    db = Pockmon(root='../data/pokeman/',resize=224,mode='train')
    x,y = next(iter(db))
    print('sample:',x.shape,y.shape,y)

    viz.image(db.denormalize(x), win='sample_x', opts=dict(title='sample_x'))

    loader = DataLoader(db
                        , batch_size=32
                        , shuffle=True
                        , num_workers=8 # 8线程去取一个batch里的图片
                        )
    for x,y in loader:
```

```
        viz.images(db.denormalize(x), nrow=8, win='batch', opts=dict(title='batch'))
        viz.text(str(y.numpy()), win='label', opts=dict(title='batch-y'))
        time.sleep(10)



    # python -m visdom.server
if __name__ == '__main__':
    main()

```

1.2 其他形式

1.2.1 train+vild分开

- 结构如下图,train和valid数据集已经分开，其下的目录即是数字标签，数字标签对应的类别名称在cat_to_name.json文件中。



- 代码

```
import torch
from torch import nn
from torch.nn import functional as F
from torchvision import datasets,models,transforms
from torch.utils.data import Dataset,DataLoader
import os,glob
import random,csv
import time
import torchvision
```

```python
from PIL import Image
import json
import visdom

class MyData(Dataset):
    def __init__(self,img_root,label_file,resize,mode):
        super(MyData,self).__init__()

        self.img_root   = img_root
        self.label_file = label_file
        self.resize     = resize
        if mode in ['train','valid']:
            self.mode   = mode
        else:
            raise Exception('mode 输入不正确')

        self.label2name = json.load(open(os.path.join(self.img_root,self.label_file)))

        # 加载所有的图片路径列表和图片label列表
        self.img_path_list,self.img_label_list = self.load_csv(f'image_path_label_{self.mode}.csv')


    def load_csv(self,filename):
        if not os.path.exists(os.path.join(self.img_root,filename)):
            img_path_list = []
            for name in self.label2name.keys():
                img_path_list += glob.glob(os.path.join(self.img_root,self.mode,name,'*.png'))
                img_path_list += glob.glob(os.path.join(self.img_root,self.mode,name,'*.jpg'))
                img_path_list += glob.glob(os.path.join(self.img_root,self.mode,name,'*.jpeg'))

            random.shuffle(img_path_list)

            # 存入csv文件
            with open(os.path.join(self.img_root,filename),mode='w',newline='') as f:
                writer = csv.writer(f)
                # '/Users/mengzhichao/Desktop/pytorch/data/flower_data/train/23/image_03395.jpg'
                for img_path in img_path_list:
                    label = img_path.split(os.sep)[-2]

                    writer.writerow([img_path,label])

```

```python
                print('图片地址列表，label列表存入：',os.path.join(self.img_root,filename))

        # 读取csv文件
        img_path_list,img_label_list = [],[]
        with open(os.path.join(self.img_root,filename)) as f:
            reader = csv.reader(f)
            for row in reader:
                img_path,label = row
                label = int(label)

                img_path_list.append(img_path)
                img_label_list.append(label)
        assert len(img_path_list) == len(img_label_list)
        return img_path_list,img_label_list

    def __len__(self):
        return len(self.img_label_list)

    def __getitem__(self,idx):
        img_path,label = self.img_path_list[idx],self.img_label_list[idx]
        tf = transforms.Compose([
            lambda x:Image.open(x).convert('RGB'),
 transforms.Resize((int(self.resize*1.25),int(self.resize*1.25))),
            transforms.RandomRotation(15),
            transforms.CenterCrop(self.resize),
            transforms.ToTensor(),
            transforms.Normalize(mean=[0.485,0.456,0.406]
                                 ,std =[0.229,0.224,0.225]
                                 )
        ])
        img = tf(img_path)
        label = torch.tensor(label)
        return img,label

    def denormalize(self,x_hat):
        """denormalize:   将已经normalize的图片tensor复原
        """
        mean=[0.485,0.456,0.406]
        std=[0.229,0.224,0.225]

        # x:[c,h,w]
        # mean: [3]=>[3,1,1]
        mean = torch.tensor(mean).unsqueeze(1).unsqueeze(1)
        std = torch.tensor(std).unsqueeze(1).unsqueeze(1)
        x = x_hat * std + mean

        return x
```

- 测试train

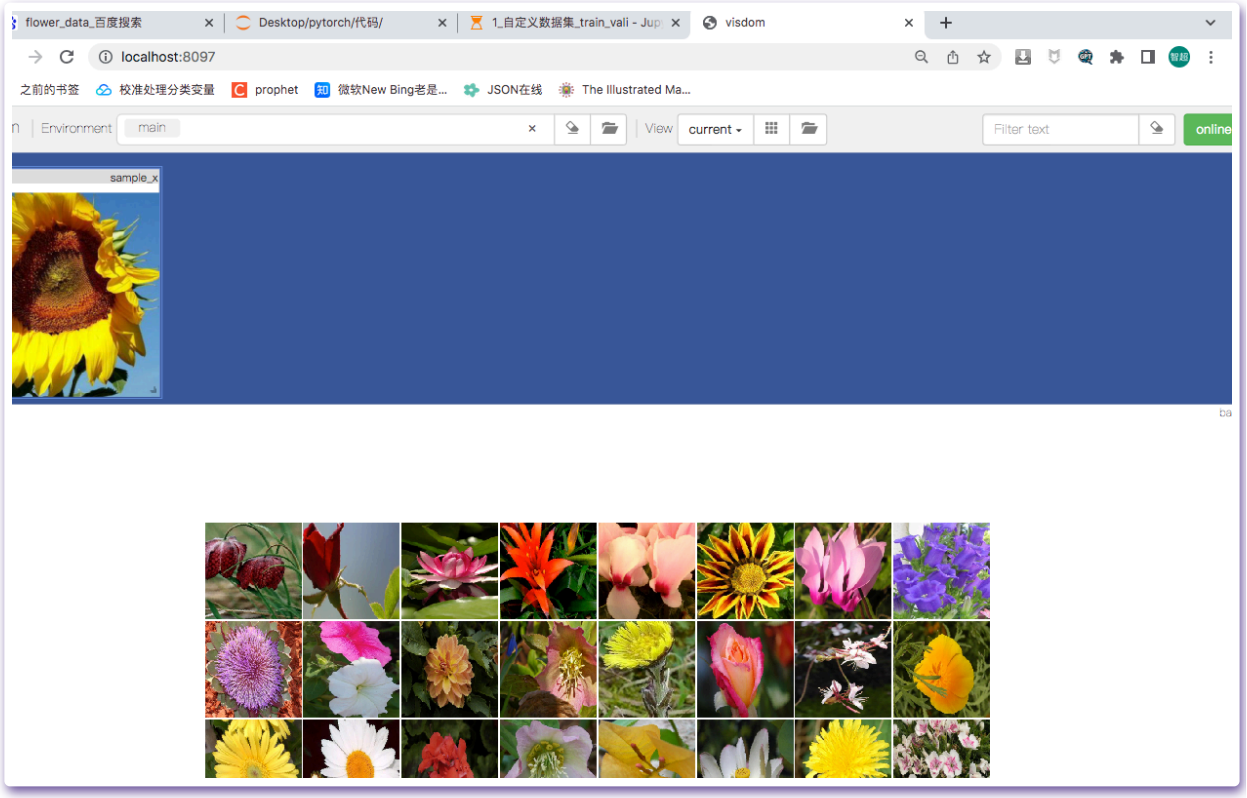
```
mydata = MyData(img_root='/Users/mengzhichao/Desktop/pytorch/data/flower_data'
                ,label_file='cat_to_name.json'
                ,resize=28
                ,mode='train'
                )
x,y = next(iter(mydata))
print('sample:',x.shape,y.shape,y)
print(len(mydata))


"""输出"""
图片地址列表，label列表存入：/Users/mengzhichao/Desktop/pytorch/data/flower_data/image_path_label_train.csv

sample: torch.Size([3, 28, 28]) torch.Size([]) tensor(23)

6552
```

- 测试valid

```
mydata = MyData(img_root='/Users/mengzhichao/Desktop/pytorch/data/flower_data'
                ,label_file='cat_to_name.json'
                ,resize=28
                ,mode='valid'
                )
x,y = next(iter(mydata))
print('sample:',x.shape,y.shape,y)
print(len(mydata))


"""输出"""
图片地址列表，label列表存入：/Users/mengzhichao/Desktop/pytorch/data/flower_data/image_path_label_valid.csv

sample: torch.Size([3, 28, 28]) torch.Size([]) tensor(54)

818
```

- visdom 测试

> 打开visdom服务：python -m visdom.server

```
viz = visdom.Visdom()
```

```
mydata = MyData(img_root='/Users/mengzhichao/Desktop/pytorch/data/flower_data'
                ,label_file='cat_to_name.json'
                ,resize=224
                ,mode='valid'
                )
loader = DataLoader(mydata
                    , batch_size=32
                    , shuffle=True
#                     , num_workers=8 # 8线程去取一个batch里的图片
                    )
for x,y in loader:
    viz.images(mydata.denormalize(x), nrow=8, win='batch', opts=dict(title='batch'))
    viz.text(str(y.numpy()), win='label', opts=dict(title='batch-y'))
    time.sleep(5)
```